

Sistemas de Informação



Disciplina: Programação I

Prof. Renê Rodrigues Veloso

Relacionamento entre classes

Objetos de software interagem e se comunicam com outros através de **mensagens**.

Quando o objeto A deseja que o objeto B execute um de seus métodos, o objeto A envia uma mensagem ao objeto B.



Relacionamento entre classes

Uma mensagem é formada por três componentes básicos:

- O objeto receptor
- O nome do método que se deseja executar
- Os parâmetros (se existirem) necessários ao método.

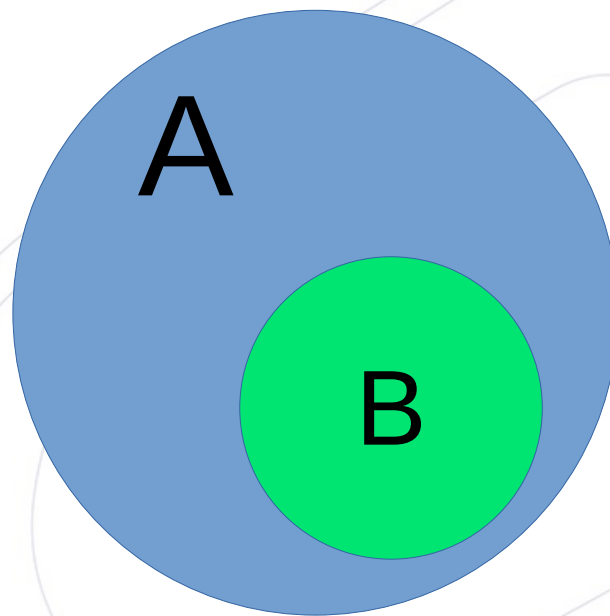


Relacionamento entre classes

... além disso, objetos conter métodos com outros objetos em seus parâmetros ou podem ter outros objetos como atributos.

O que indica um tipo de relacionamento TODO-PARTE, que pode ser de:

- associação
- agregação
- composição



Associação (Dependência)

Relacionamento mais fraco de todos!

Indica uma relação entre os objetos (momentânea ou não).

Exemplo:

```
public class A{  
    ...  
  
    public A(){ ... }  
  
    public void metodo(B b){  
        b.metodo_de_b();  
    }  
}
```

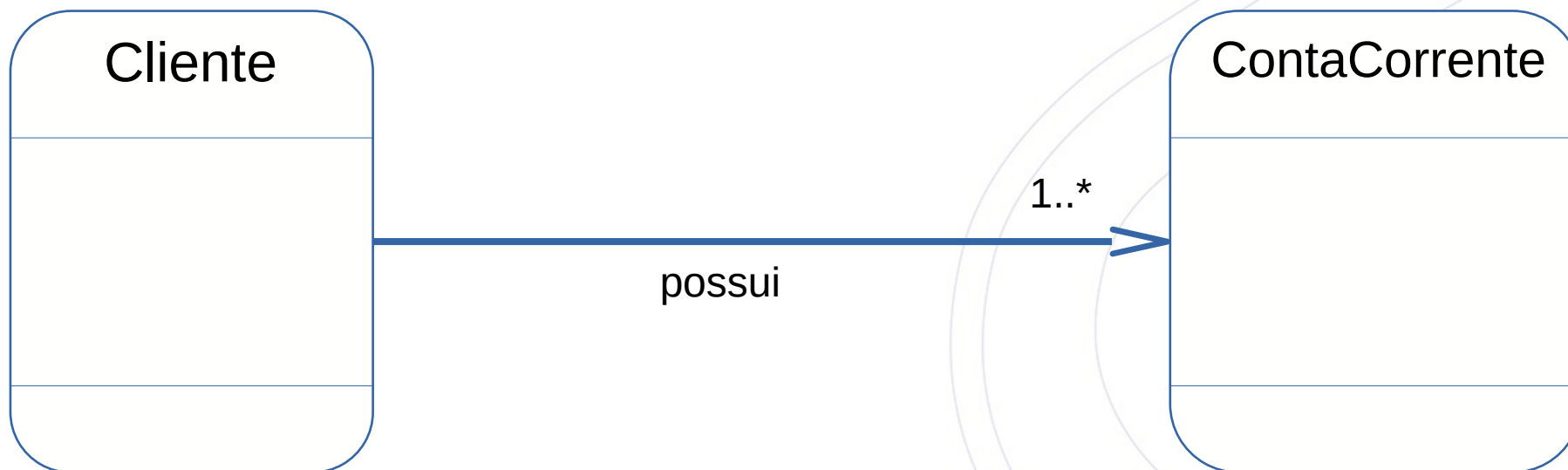
```
public class A{  
    private atributo de B;  
  
    public A(){ ... }  
  
    public void metodo(B b){  
        copia um atributo de b  
    }  
}
```

Associação (Dependência)

Relacionamento mais fraco de todos!

Indica uma relação entre os objetos (momentânea ou não).

Representação: uma linha conectando os dois objetos (pode existir uma seta ao final, apontando para o objeto que está sendo usado). Pode também receber um nome e uma cardinalidade.

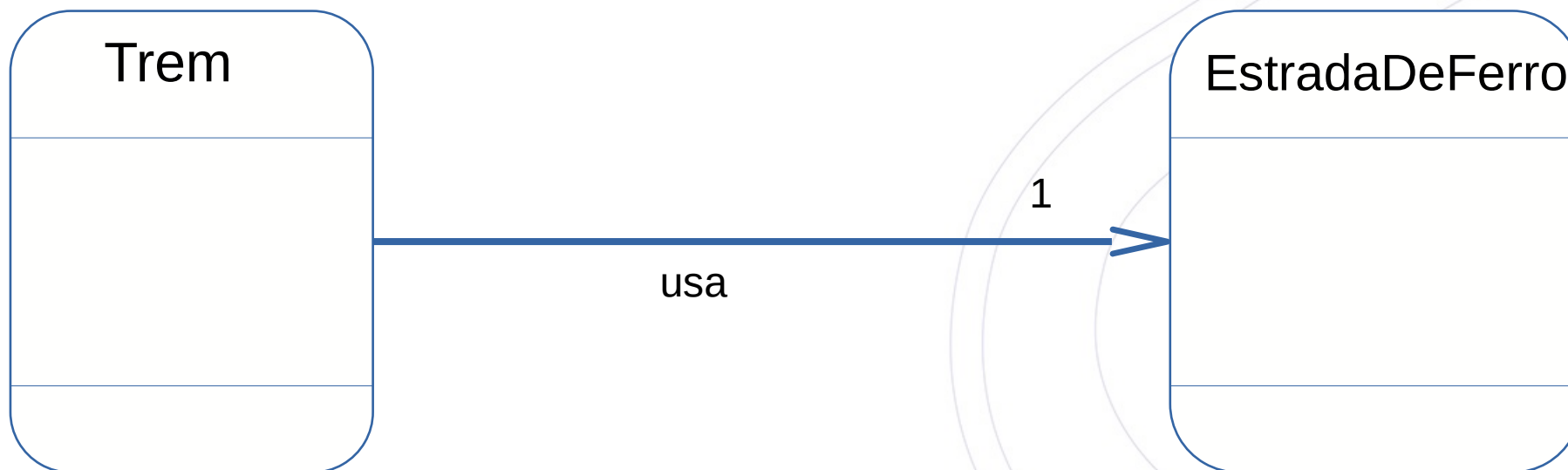


Associação (Dependência)

Relacionamento mais fraco de todos!

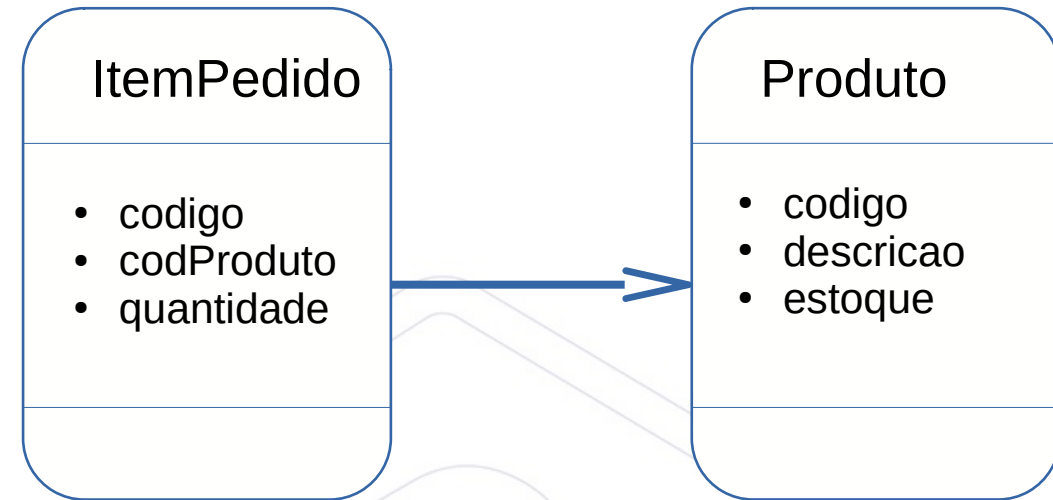
Indica uma relação entre os objetos (momentânea ou não).

Representação: uma linha conectando os dois objetos (pode existir uma seta ao final, apontando para o objeto que está sendo usado). Pode também receber um nome e uma cardinalidade.



Associação (Dependência)

```
class ItemPedido {  
    private int codigo;  
    private String codProduto;  
    private int quantidade;  
  
    public ItemPedido(String codProd){  
        this.codProduto = codProd;  
        ...  
    }  
    ...  
}
```

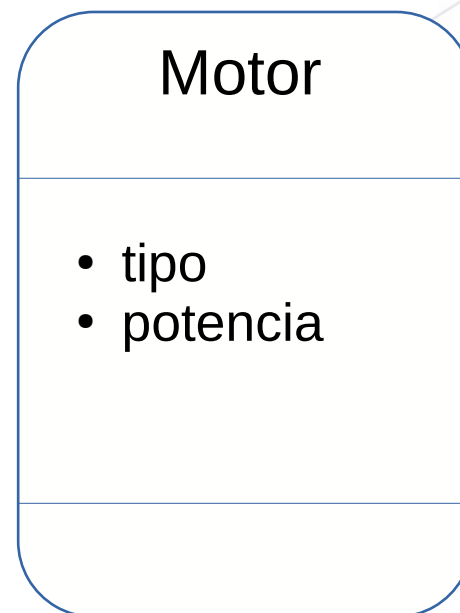
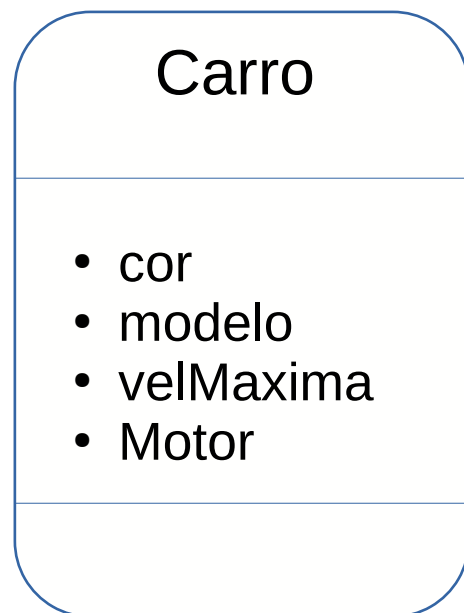


Agregação

Relacionamento **todo-parte** mais forte de que **associação**;

Indica uma relação duradoura entre os objetos, **na qual as partes são independentes do todo**.

Exemplo: carro e motor

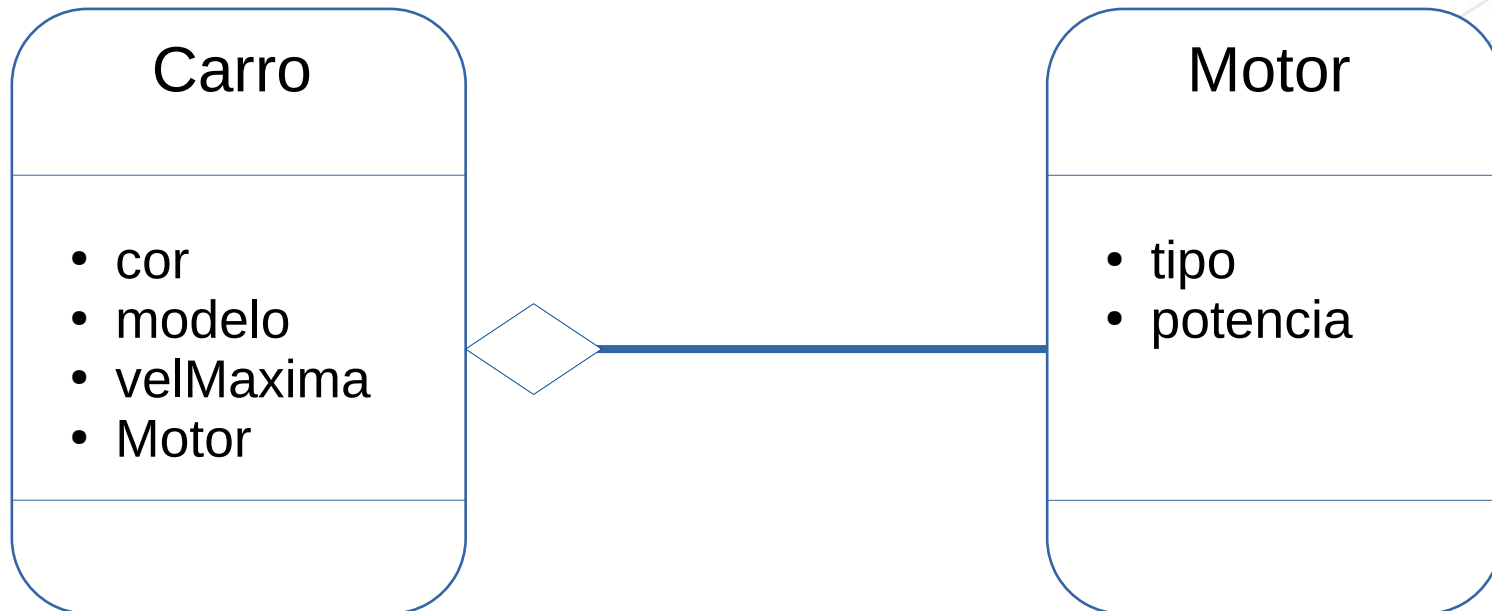


Agregação

Relacionamento **todo-parte** mais forte de que **associação**;

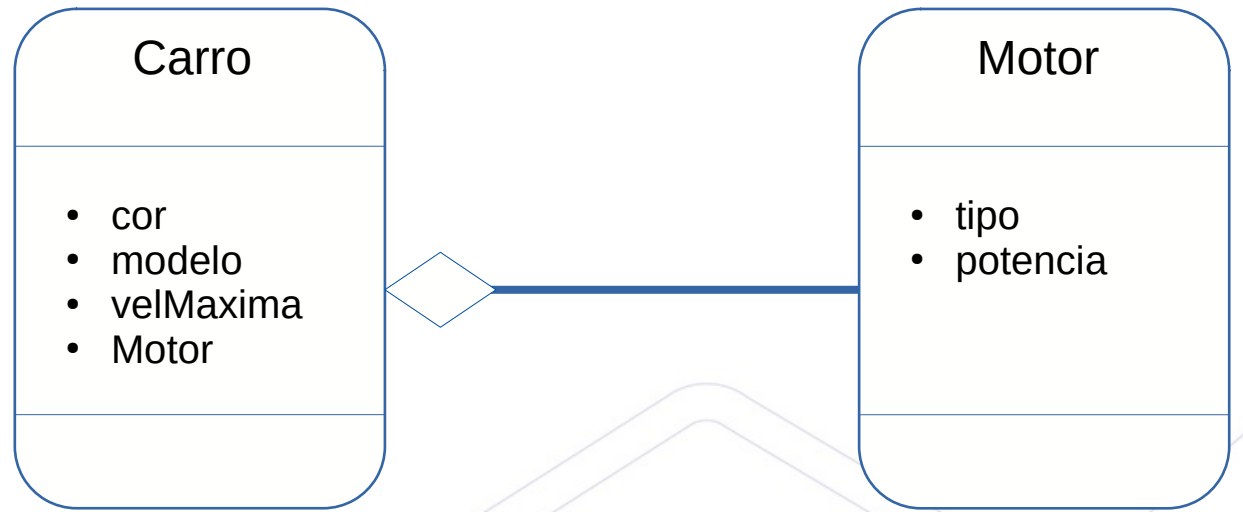
Indica uma relação duradoura entre os objetos, **na qual as partes são independentes do todo**.

Representação: losango vazio junto ao objeto representando o todo.



Agregação

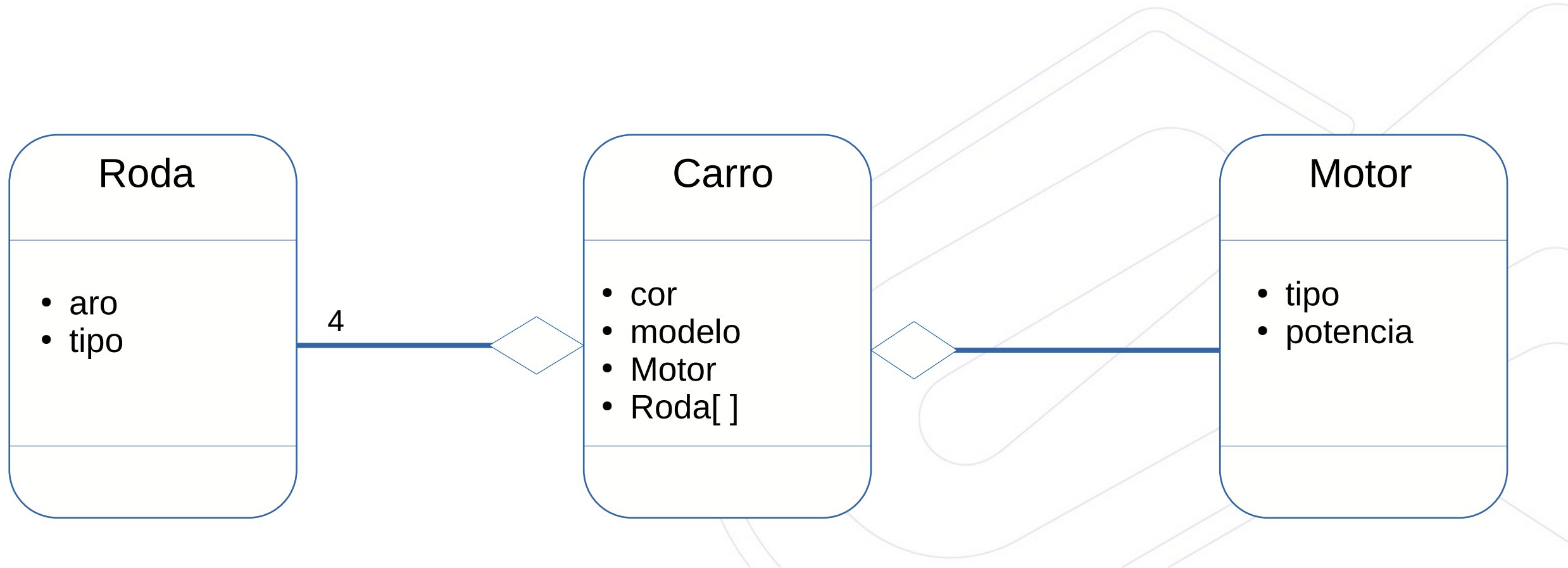
```
class Carro {  
    private String cor;  
    private String modelo;  
    private float velMaxima;  
    private Motor m;  
  
    public Carro(Motor m){  
        this.m = m;  
    }  
    ...  
}
```



Agregação

Relacionamento **todo-parte** mais forte de que **associação**;

Pode haver uma lista de partes.



Agregação

```
class Carro {
```

```
    private String cor;
```

```
    private String modelo;
```

```
    private float velMaxima;
```

```
    private Motor m;
```

```
    private Roda[] rodas;
```

```
    public Carro(Motor m, Roda r1, Roda r2, Roda r3, Roda r4){
```

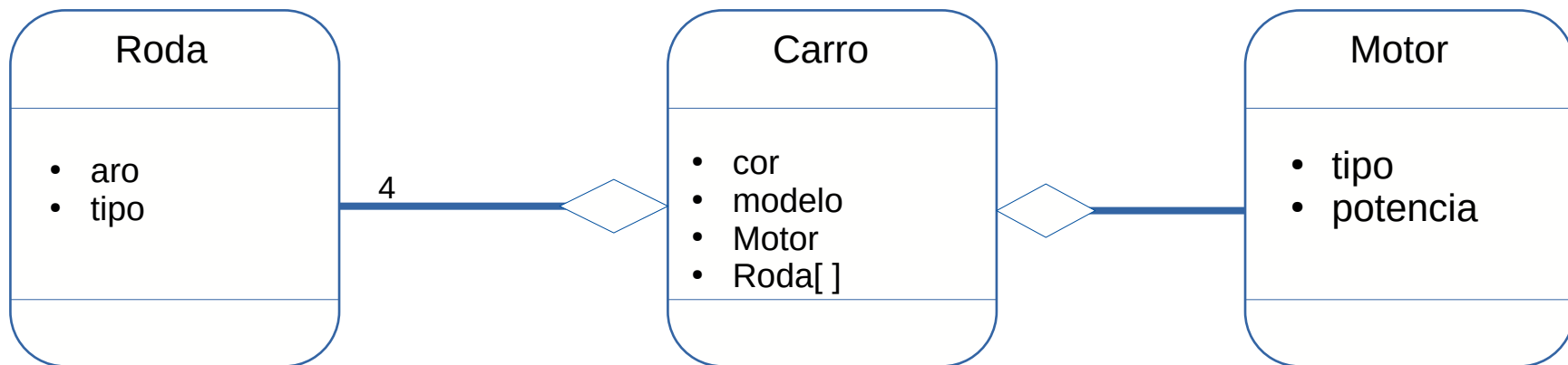
```
        this.m = m;
```

```
        rodas = {r1, r2, r3, r4};
```

```
    }
```

```
    ...
```

```
}
```

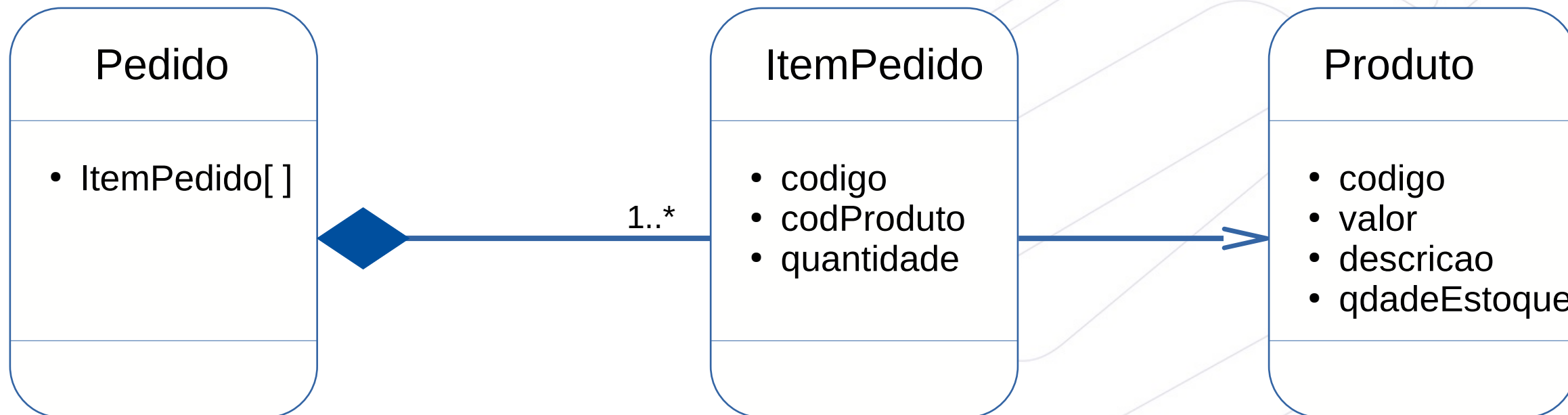


Composição

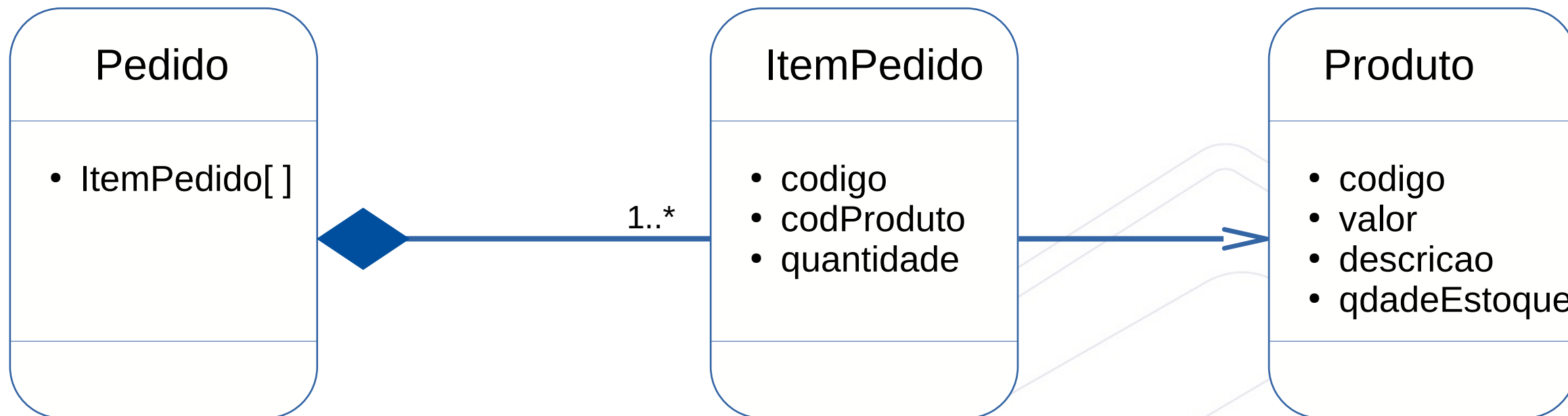
Relacionamento **todo-parte** mais forte de que **agregação**;

Indica uma relação duradoura entre os objetos, **na qual as partes são dependentes do todo**, ou seja, não existem o todo.

Representação: losango preenchido junto ao objeto representando o todo.

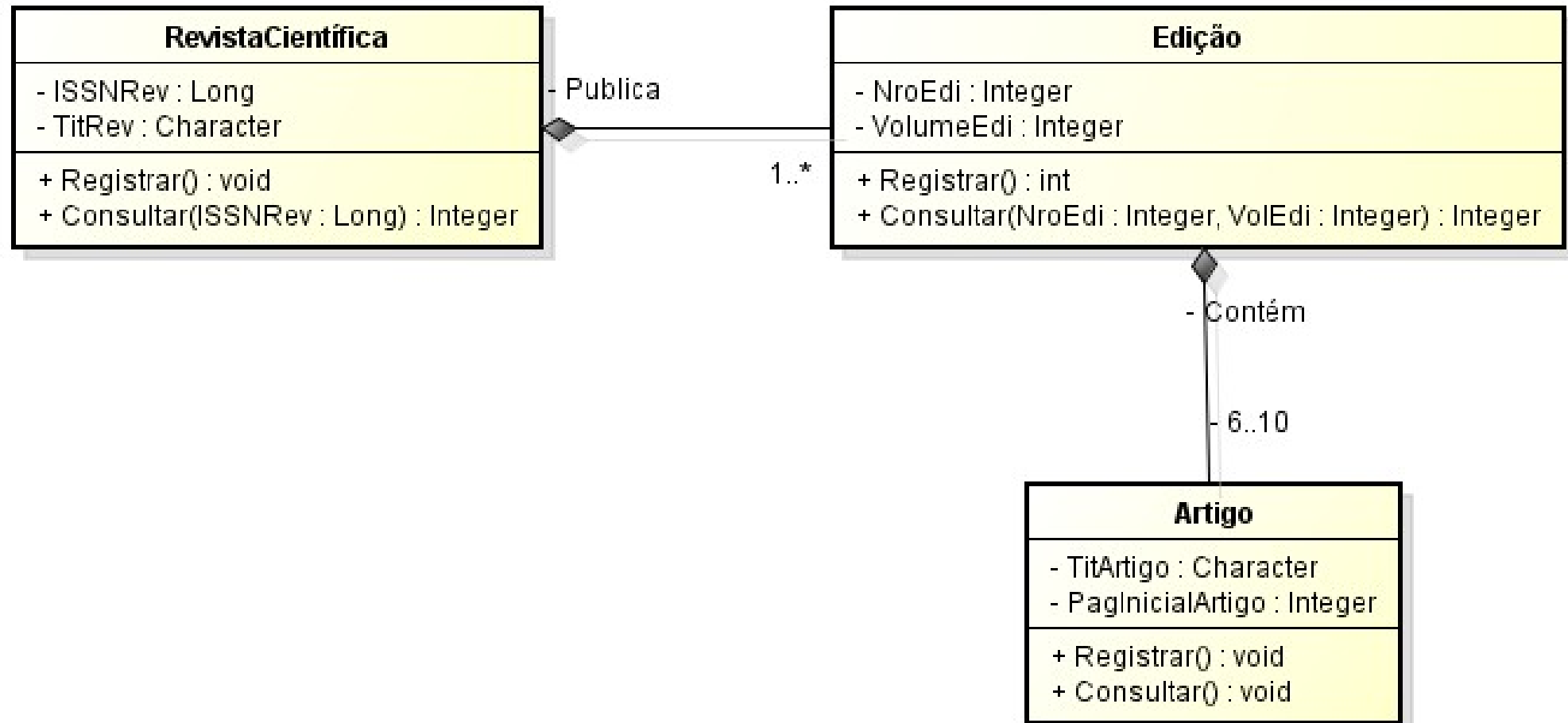


Composição



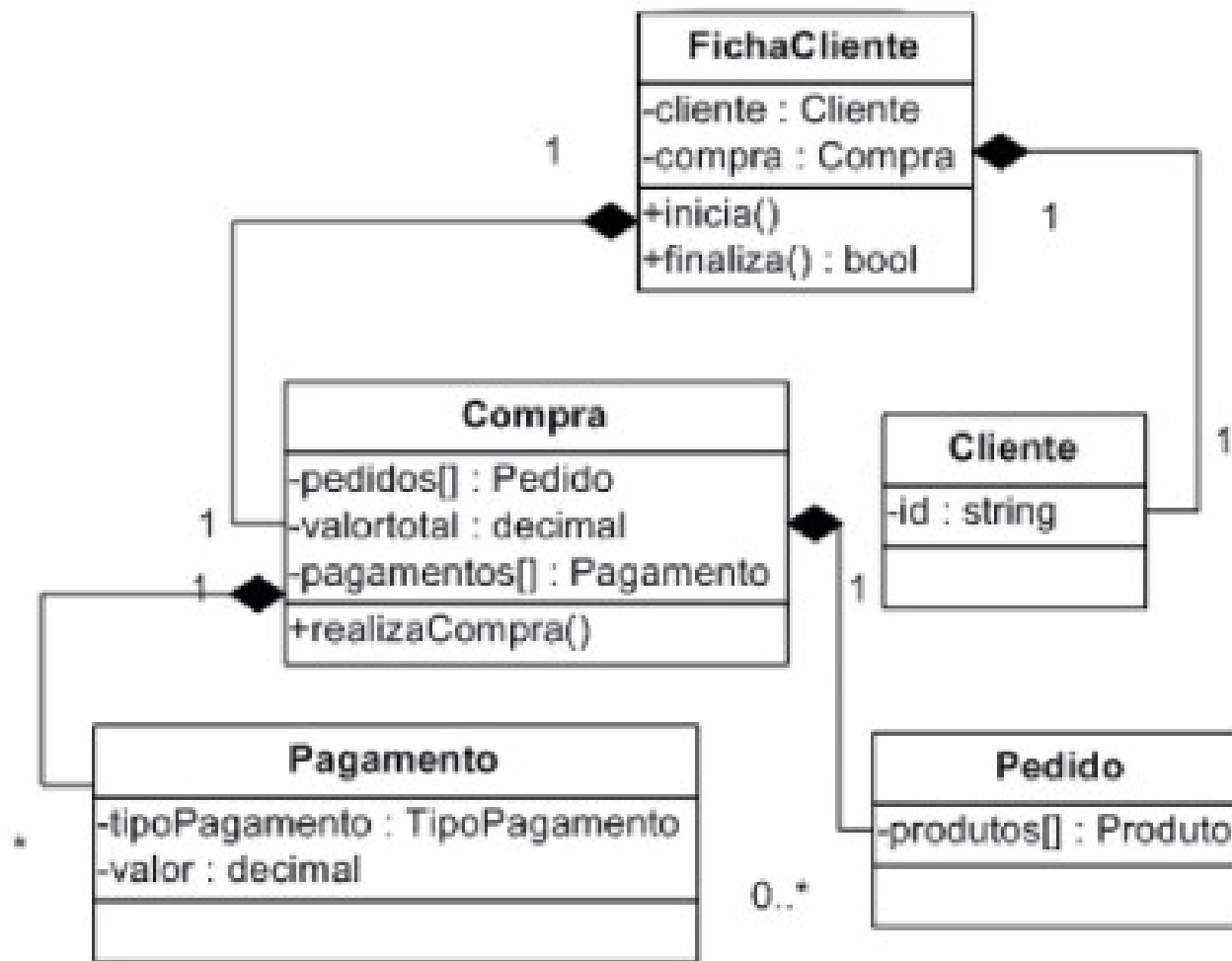
- Um pedido é **composto** por um conjunto de itens pedidos.
- Um item pedido **associa-se** com um produto por meio do seu código.

Composição



Exercício

O seguinte diagrama de classes faz sentido, falta algo?



Exercício

Apresente o diagrama de classes e a implementação em Java para:

Uma classe representando um ponto em um plano bidimensional (x,y), com seus atributos e métodos.

Uma classe representando um retângulo, com um ponto de origem (objeto da classe ponto), uma altura e uma largura.

- indique métodos para manipular apropriadamente os atributos, incluindo a modificação da origem (mover o retângulo) e para calcular a sua área
- indique um construtor para um retângulo em um ponto específico, altura e largura
- indique um construtor para um retângulo na origem (0,0)

Exercício

Apresente o diagrama de classes e a implementação em Java para:

Uma classe para um Cartão de Crédito: `CartaoCredito`. Atributos: número, nome do titular, nome do banco, saldo, limite.

Uma classe representando uma carteira com até 3 cartões. Deve conter métodos para manipular os cartões.

Exercício

Apresente o diagrama de classes e a implementação em Java para:

Uma classe Agenda que pode armazenar informações de até 10 pessoas e que seja capaz de realizar as seguintes operações:

- `void armazenaPessoa(String nome, int idade, float altura);`
- `void removePessoa(String nome);`
- `int buscaPessoa(String nome);` // informa em que posição da agenda está a pessoa
- `void imprimeAgenda();` // imprime os dados de todas as pessoas da agenda
- `void imprimePessoa(int index);` // imprime os dados da pessoa que está na posição "i" da agenda.

Exercício

Escreva uma classe Data cuja instância (objeto) represente uma data. Esta classe deverá dispor dos seguintes métodos:

- construtor: define a data que determinará o objeto (através de parâmetro), este método verifica se a data está correta, caso não esteja a data é configurada como 01/01/0001
- compara: recebe como parâmetro um outro objeto da Classe data, compara com a data corrente e retorne:
 - 0 se as datas forem iguais;
 - 1 se a data corrente for maior que a do parâmetro;
 - -1 se a data do parâmetro for maior que a corrente.
- getDia: retorna o dia da data
- getMes: retorna o mês da data
- getMesExtenso: retorna o mês da data corrente por extenso
- getAno: retorna o ano da data
- clone: o objeto clona a si próprio, para isto, ele cria um novo objeto da classe Data com os

Exercício

Escreva uma classe em que cada objeto represente um voo que acontece em determinada data e em determinado horário. Cada voo possui no máximo 100 passageiros, e a classe permite controlar a ocupação das vagas. A classe deve ter os seguintes métodos:

- construtor: configura os dados do voo (recebidos como parâmetro): número do voo, data (para armazenar a data utilize um objeto da classe Data, criada na questão anterior);
- proximoLivre: retorna o número da próxima cadeira livre
- verifica: verifica se o número da cadeira recebido como parâmetro está ocupada
- ocupa: ocupa determinada cadeira do voo, cujo número é recebido como parâmetro, e retorna verdadeiro se a cadeira ainda não estiver ocupada (operação foi bem sucedida) e falso caso contrário
- vagas: retorna o número de cadeiras vagas disponíveis (não ocupadas) no voo
- getVoo: retorna o número do voo
- getData: retorna a data do voo (na forma de objeto)
- clone: o objeto clona a si próprio, para isto, ele cria um novo objeto da mesma classe e faz uma cópia dos valores de seus atributos